Memory model: image, handles, views, and actor epochs

Purpose

Define the vocabulary and the target model for memory in Scale. The immediate scope is Elixir/BEAM compatibility. The Go transpiler is delayed and is not a driver for this model.

The model keeps the append-only image as the value authority. It adds a private lifetime region per actor incarnation only where actor memory behavior requires one. It does not add a global mutable heap.

Terms

Term	Definition	Scale use
Stack	Last-in, first-out execution memory for operands, calls, and continuations	The evaluation and return stacks
Image / store	Persistent, identity-addressed, append-only value authority	`Store`, `Lexicon`, `TextCache`, `EntryIdentity`
Arena	A region whose values share one bulk lifetime and are released together	An actor epoch is a form of arena
Epoch	A linear arena lifetime with exactly one close	Per actor incarnation, not per message
Handle	A stable value that names an arena or image block	`TensorHandle` would carry arena, offset, length, rank, and generation
View	A stack-only projection over a handle or image block	`TensorView` would carry `ReadOnlySpan<double>` and `ReadOnlySpan<int>`
Classic heap	Unrestricted mutable object graph with aliasing and non-stack lifetime	Not part of the semantic model
BEAM process heap	A private per-process term region with per-process GC and copy-on-send	Not implemented
CLR managed heap	The .NET runtime's GC implementation for managed objects and arrays	Host detail; current `double[]` payloads live there
CLI	The specification that defines the .NET type system and execution model	Not a heap; the CLR implementation supplies the managed heap

The important distinction is that "heap" can mean a semantic model, a runtime implementation, or a lifetime region. Scale uses none of the first two as a semantic model. It does need dynamic allocation, and it will need an explicit actor lifetime region for operational Elixir compatibility.

Current state

- `Tensor` is a host record with `Data: double[]` and `Shape: int[]` (`types.fs`). Both are CLR-managed arrays.
- `Store` is one shared append-only `Map<EntryIdentity, Tensor>` per run.
- `switchTo` threads the same live store into every actor (`scheduler.fs`).
- Mailboxes and channels are store-resident; messages are queued as identities.
- `OwnershipState` is per-run execution state and is never serialized.
- `Reclaim.report` treats numeric values as possible references; with more than one actor every entry is treated as a root, so compaction keeps the image.

- There is no per-actor allocation region, no promotion boundary, and no per-process GC.

This is a shared immutable value store. It is not a classic heap, but it is also not a private heap per actor.

Decision

1. The image remains the authority for durable and cross-actor values.
2. Ephemeral actor values may live in a per-incarnation epoch.
3. A value crossing an epoch boundary is promoted to the image, or copied into the receiver's epoch. It must never leave a dangling reference into the sender's epoch.
4. `EntryIdentity` stays a struct. Handles and metadata may use `[<Struct>]` as a host performance choice.
5. A tensor view may use `[<Struct; IsByRefLike>]` as a transient host view.
6. `Tensor` is not a `ref struct`; it cannot be stored in a class, `Map`, actor state, mailbox, closure, or persistence.
7. The first arena backend uses managed storage. Unmanaged storage is a later decision, not part of the first step.
8. Promotion and finalization are separate. Memory reclamation closes epochs; side-effect finalization uses a linear token and `bracket`.
9. There is no global mutable heap. There is no per-process tracing GC in this design.

Struct annotations are not semantics. They are host performance choices and a worked example of the host/semantics split. They can reduce wrapper allocation, boxing, and copies; they do not create ownership, lifetime, aliasing, promotion, or reclamation rules. A struct can still contain aliased arrays, and a byref-like view can still point at live memory.

Frozen (D90). The representation is frozen for the port: managed backing; shared immutable values with promotion at every escape; no copy-on-send; the evaluator is not routed through the arena. The image remains the authority.

Why handles and views are split

A handle is storable and stable:

```
[<Struct>]
type TensorHandle =
    { Arena: int
      Offset: int
      Length: int
      Rank: int
      Generation: int }
```

A view is stack-only and transient:

```
[<Struct; IsByRefLike>]
type TensorView =
    { Data: ReadOnlySpan<double>
      Shape: ReadOnlySpan<int> }
```

`TensorHandle` can live in `Store`, `EvaluationState`, an actor, a mailbox, or a persisted record. `TensorView` cannot. It is created for an operation and dies before the next suspension point.

Making `Tensor` itself a `[<Struct>]` removes only the wrapper allocation. Its `double[]` fields remain CLR-managed. Making `Tensor` a `ref struct` would make it impossible to store in the current image and actor model.

These annotations are host-performance choices and a worked example of the split, not semantic guarantees. The semantic rules live in the image authority, ownership, promotion, and reclamation decisions; the annotations only make the host representation cheaper and more legible.

Non-goals

- A classic mutable heap.
- A global shared object graph.
- A full per-process tracing GC at this stage.
- Unmanaged arenas at this stage.
- A `ref struct Tensor` in persistent state.
- Elixir distribution or remote code sharing.
- The Go transpiler.

Staging

Stage	Work	Acceptance
0	Define terms, ownership, and the handle/view split	This document and D80 compile; no runtime behavior changes
1	Introduce `TensorHandle`, `TensorView`, and the arena contract	A handle round-trips; a view cannot escape; tensor operations preserve shape
2	Add a per-incarnation actor epoch and promotion boundaries	A sender's termination cannot invalidate a receiver's message; persistence promotes
3	Add per-actor reclamation and compaction	A long-lived actor releases unreachable epochs; global roots survive
4	Decide copy-on-send and optionally an unmanaged arena backend	Elixir message isolation holds; NativeAOT and safety tests pass
5	Consider a per-process tracing GC only for exact operational BEAM parity	A separate runtime decision, not implied by stages 0–4

Status (2026-10-31). Stage 1 is D89, stages 2–3 are D85–D88, and stage 4 is decided by D90 (managed, shared immutable, no copy-on-send). Stage 5 remains optional.

Integration rules

- The image is append-only; never rewrite history.
- Execution state is never serialized; roots are re-threaded by the reader.
- An ephemeral identity never enters `Lexicon`, `TextCache`, a mailbox, a persisted record, or another actor's stack without promotion.
- A mutable arena is a runtime resource, not a field inside an immutable `EvaluationState` snapshot.
- A per-message arena is invalid because `yield` and `halt` can suspend inside a handler. The default is per incarnation.
- A `ref struct` view must not cross a suspension, closure, `Task`, or persistence boundary.
- Ownership stays dynamic until a static contract is strong enough to replace it; `Undecided` is never a pass.
- Finalizers protect side effects; epochs protect memory. Do not conflate them.
- The image remains the authority for distributed or persisted values; remote serialization will reuse the promotion boundary.

Open decisions

- Epoch lifetime: per incarnation (D85).
- Message policy: shared by reference with promotion at send (D86); no copy-on-send (D90).
- Backend: managed storage first (D80, D90); unmanaged remains deferred.
- Reclamation: closed-epoch candidates intersected with live roots (D88).
- Enforcement: dynamic ownership plus the conservative static trace checker (D30, D54, D81).
- Large immutable values: shared by reference (D90).